Programming
Human-Centered Artificial Intelligence
Dr. Ramon Winterhalder
A.Y. 2026/27

Exercise sheet 8

Functions

Not graded, not handed in. Do them anyway. This is where the learning happens! Solutions are published after we discuss them in class.

Docstring, tests, body. In that order. For every function you write on this sheet, write the docstring first. Then write three calls with the answers you expect. Only then write the body, and run the calls. The exam asks you to write a function, and this is how to get it right on paper.

Part A — Defining and calling

A1 *Print or return?*

Each program runs as a script. Say exactly what it shows.

```
# (1)
def shout(word):
    print(word.upper() + "!")

r = shout("hi")
print(r)

# (3)
def first(xs):
    return xs[0]
    print("found it")

print(first([7, 8]))
```

```
# (2)
def half(x):
    return x / 2

half(10)
print(half(half(10)))

# (4)
def sign_word(n):
    if n > 0:
        return "positive"

print(sign_word(5), sign_word(-5))
```

In (2), the line `half(10)` shows nothing in a script. What would it show as the last line of a notebook cell, and why the difference?

A2 *They run, and they are wrong*

Each function runs without an error and gives a wrong answer for some input. For each, give a call that shows the bug, say what is wrong, and fix it.

```
# (1)
def is_even(n):
    print(n % 2 == 0)

if is_even(4):
    print("even")
else:
    print("odd")

# (2)
def contains(xs, target):
    for x in xs:
        if x == target:
            return True
        else:
            return False

# (3)
def total(xs):
    t = 0
    for x in xs:
        t += x
        return t

# (4)
def area(w, h):
    w * h
```

Number 2 is the one most people write at least once. Which call makes it fail, and which call happens to work?

A3 *Two functions from their descriptions*

Docstring first, then three test calls, then the body.

a) `clamp(x, low, high)` gives back `x`, but never less than `low` and never more than `high`.

b) `median(xs)` gives back the middle value of a list of numbers. For an even number of items it is the mean of the two in the middle. The caller's list must not change.

For part 2, check the last sentence with a test of its own.

A4 *Once, not three times*

```
title = "Results"
print("+" + "-" * (len(title) + 2) + "+")
print("| " + title + " |")
print("+" + "-" * (len(title) + 2) + "+")

title = "Summary"
print("+" + "-" * (len(title) + 2) + "+")
print("| " + title + " |")
print("+" + "-" * (len(title) + 2) + "+")

title = "Next steps"
print("+" + "-" * (len(title) + 2) + "+")
print("| " + title + " |")
print("+" + "-" * (len(title) + 2) + "+")
```

a) Write a function `boxed(title)` and replace the twelve lines by three calls.

b) The corners should now be `*` instead of `+`. How many places do you change before, and how many after?

c) This function prints and returns nothing. Is that a problem here? Say why not.

Part B — Arguments and scope

B1 *Defaults and keywords*

```
def describe(name, age=18, city="Milano"):
    return f"{name}, {age}, {city}"
```

Give the value of each call, or the error it causes. Three of them are broken, and one of those three runs without complaint.

```
1.  describe("Ada")                    5.  describe("Ada", "London")
2.  describe("Ada", 36)                6.  describe(city="Roma")
3.  describe("Ada", city="London")     7.  describe(name="Ada", 36)
4.  describe(age=20, name="Alan")
```

B2 *Who can see what*

```
x = 5

def f():
    x = 10
    return x

def g():
    return x + 1

def h(x):
    x = x * 2
    return x

print(f(), x)
print(g())
print(h(x), x)
```

a) Give the three lines this prints.

b) This one stops with an error. Which, and why?

```
total = 0

def add(n):
    total = total + n

add(3)
```

c) Fix it without the word `global`. The function should receive the old total and give back the new one.

B3 *Whose list is it?*

Each of these is called as `func(nums)`, starting every time from a fresh `nums = [1, 2, 3]`. Give `nums` after each call.

```
def a(xs):                  def b(xs):
    xs.append(0)                xs = xs + [0]

def c(xs):                  def d(xs):
    xs[0] = 99                  xs = []
```

Now this function, meant to remove the failed grades from a list:

```
def remove_failed(grades):
    for g in grades:
        if g < 18:
            grades.remove(g)

gs = [15, 12, 28, 17, 30]
remove_failed(gs)
print(gs)
```

a) Predict what it prints, then check. Explain the result.

b) Write `passed_only(grades)`, which leaves `grades` alone and gives back a new list.

c) Fix `remove_failed` so that it does change `grades` in place, correctly. One small change to the `for` line is enough.

B4 *A weather report, top down*

```
temps = [14.5, 17.0, 21.5, 19.0, 12.5, 16.0, 22.5]     # Monday to Sunday
```

The report prints the lowest, highest and mean temperature, and the numbers of the days warmer than 20 degrees, counting Monday as day 1.

a) Write `main` first, calling `summary(temps)` and `warm_days(temps)`, which do not exist yet.

b) `summary` gives back the three numbers as one tuple.

c) `warm_days` has a parameter `threshold` with default 20. Call it a second time with `threshold=18`.

If you finish early

Write `apply_to_all(f, xs)`, which gives back a new list with `f` applied to every item of `xs`. Then use it three times: with `len` on a list of words, with `str.upper` on a list of names, and with a function of your own.
You have just written something Python already has, under the name `map`.